



User Interface in Android

A Simple, Easy-to-Understand Study Guide

Topics Covered:

Layouts • Fragments • Views • Adapters

Intents & Broadcast Receivers

Downloading Internet Resources • Download Manager • Internet Services

Table of Contents

1. Building User Interfaces – Introduction to Layouts
2. Introduction to Fragments
3. Creating New Views
4. Introduction to Adapters
5. Intents and Broadcast Receivers – Introduction to Intents
6. Creating Intents and Broadcast Receivers
7. Using Internet Resources – Downloading and Parsing
8. Using the Download Manager
9. Using Internet Services

1. Building User Interfaces — Introduction to Layouts

The **User Interface (UI)** is everything the user sees and touches on the screen — buttons, text, images, menus. In Android, we design the UI using something called a **Layout**.

TECHNICAL TERM

A **Layout** is an invisible container that defines how the visible elements (called **Views** — like buttons, text boxes) are arranged on the screen — one below another, side by side, or in a grid.

SIMPLE ANALOGY

Think of a layout like a **plate arrangement at a dinner table**. The plate (layout) decides where the spoon, fork, and glass (views/buttons/text) are placed — in a line, in a circle, or stacked.

Layouts are written in **XML** files and stored inside the `res/layout/` folder of your project.

Types of Layouts in Android

Layout Type	Simple Meaning	When to Use
LinearLayout	Places views one after another in a single line	Vertical or horizontal list of items
RelativeLayout	Places views relative to each other or to the parent	"Put this button below that text box"
ConstraintLayout	Places views using flexible constraints (rules), works well for complex screens	Modern, most recommended layout for any UI
FrameLayout	Stacks views on top of each other, like a pile of cards	Showing one view on top of another (e.g. image with a badge)
TableLayout	Arranges views in rows and columns, like a table	Grid-like data display

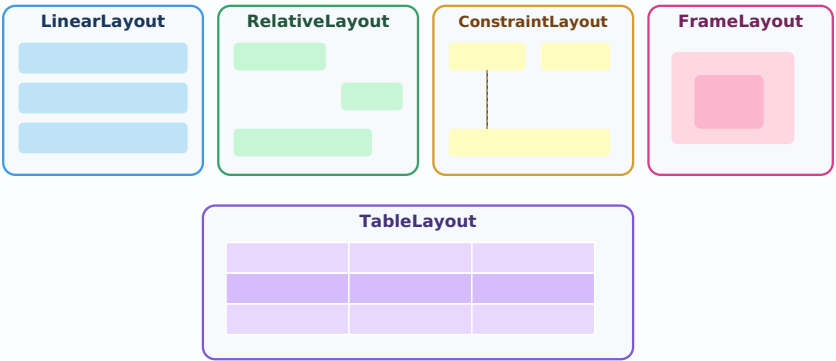


Fig 1.1 — Common Android Layout Types and how they arrange elements

DIAGRAM EXPLAINED

- **LinearLayout box** — the blue strips are stacked one below another, showing items placed in a single straight line.
- **RelativeLayout box** — the green blocks are placed at different positions relative to each other, not in one fixed line.
- **ConstraintLayout box** — the yellow blocks are connected using invisible "constraint lines" (dashed line shown) that decide their position.
- **FrameLayout box** — the pink blocks are stacked directly on top of one another, like layers of a card.
- **TableLayout box** — the purple cells are arranged in neat rows and columns, just like a spreadsheet.

Example: A Simple LinearLayout

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:text="Welcome to My App" />

        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Click Me" />

    </LinearLayout>
```

Here, `android:orientation="vertical"` means the TextView and Button will be stacked one below another (top to bottom). If we set it to `horizontal`, they would appear side by side.

EXAMPLE

A login screen usually uses a **LinearLayout (vertical)** — Username box, then Password box, then Login button, one below the other, just like filling a paper form from top to bottom.

Common View Attributes used inside Layouts

Attribute	Meaning
<code>layout_width</code> / <code>layout_height</code>	How wide/tall the view should be (match_parent, wrap_content, or a fixed size)
<code>orientation</code>	Direction of arrangement — vertical or horizontal
<code>gravity</code>	Aligns content inside a view (left, right, center)
<code>padding</code> / <code>margin</code>	Space inside/outside a view

EXAM TIP

Padding = space INSIDE the box (between border and content). **Margin** = space OUTSIDE the box (between the box and other elements).

2. Introduction to Fragments

TECHNICAL TERM

A **Fragment** is a reusable, independent piece (a "mini-screen" or sub-section) of a user interface that lives inside an Activity. A Fragment has its own layout and its own lifecycle, but it must always be hosted inside an Activity — it cannot exist on its own.

SIMPLE ANALOGY

If an Activity is like a full **newspaper page**, a Fragment is like one **column or box** printed on that page. You can mix and match different columns on the same page, or reuse the same column design on different pages.

Why Use Fragments?

- **Reusability** – The same Fragment can be reused on multiple screens.
- **Better for Tablets** – A tablet screen is big, so you can show two Fragments side by side (e.g., email list + email content), while a phone shows them one at a time.
- **Modular Design** – Breaks a big, complex screen into smaller, manageable parts.
- **Dynamic UI** – Fragments can be added, removed, or replaced while the app is running, without starting a new Activity.

Phone (One Fragment at a time)



Tablet (Two Fragments together)

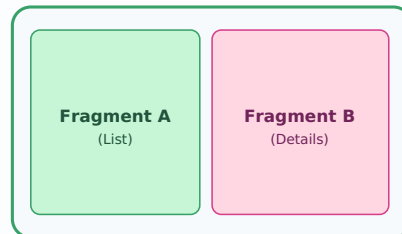


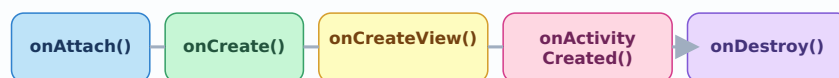
Fig 2.1 — Same Fragments reused differently on Phone vs Tablet

DIAGRAM EXPLAINED

- **Left side (Phone)** — the screen is small, so only one Fragment (Fragment A — the List Screen) is shown at a time, filling the whole Activity.
- **Right side (Tablet)** — the screen is bigger, so the same Activity can host two Fragments together side by side: Fragment A (List) on the left and Fragment B (Details) on the right.
- This shows the main benefit of Fragments — the exact same Fragment code can be arranged differently depending on how much screen space is available, without rewriting the app.

Fragment Lifecycle (Simplified)

A Fragment has its own lifecycle, similar to an Activity, but with a few extra steps because it is attached to an Activity:



Fragment is "attached" to the Activity first, and "detached" before it is destroyed

Fig 2.2 — Simplified Fragment Lifecycle

DIAGRAM EXPLAINED

- The arrows show the **order** in which Android calls these methods automatically — one after another, from left to right — as the Fragment is being set up.
- When the Fragment is removed or the Activity closes, Android calls the methods again but in the **reverse direction** to clean everything up, ending at `onDestroy()`.

Fragment Lifecycle Methods Explained

Method	When it is Called	Simple Meaning
<code>onAttach()</code>	When the Fragment is first connected to its host Activity	Fragment says "hello" and links itself to the Activity
<code>onCreate()</code>	When the Fragment itself is being created	Fragment sets up its basic data (not the screen design yet)
<code>onCreateView()</code>	When the Fragment needs to draw its UI	Fragment connects to its XML layout and builds the visible screen part
<code>onActivityCreated()</code>	After the host Activity's <code>onCreate()</code> has fully finished	Fragment can now safely use anything from the Activity
<code>onStart()</code> / <code>onResume()</code>	When the Fragment becomes visible / interactive	Fragment is now on screen and usable (similar to Activity's lifecycle)
<code>onPause()</code> / <code>onStop()</code>	When the Fragment is no longer in focus / visible	Fragment is being hidden or the user moved away from it
<code>onDestroyView()</code>	When the Fragment's screen (View) is being removed	The visible layout is cleared, but the Fragment object may still exist
<code>onDetach()</code>	When the Fragment is fully disconnected from the Activity	Fragment says "goodbye" and is no longer linked to the Activity

Example: Creating a Simple Fragment

```
public class HomeFragment extends Fragment {

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                           Bundle savedInstanceState) {
        // Connects this Fragment to its XML layout file
        return inflater.inflate(R.layout.fragment_home, container, false);
    }
}
```

EXAMPLE

In an Instagram-like app, the **bottom navigation bar** (Home, Search, Reels, Profile) switches between different **Fragments** inside the same Activity — the Activity doesn't change, only the Fragment shown inside it changes.

3. Creating New Views

TECHNICAL TERM

A **View** is the basic building block of the User Interface — it is a rectangular area on the screen responsible for drawing something and handling user interaction (like a Button, TextView, ImageView, EditText).

Android already gives us many ready-made Views (Button, TextView, ImageView, etc.). But sometimes, these ready-made views are not enough for what we want to build. In such cases, we can create a **Custom View** (a new View made by the developer).

SIMPLE ANALOGY

Ready-made Views are like **store-bought furniture** — good for most homes. But sometimes you need a **custom-made table** to fit a special corner of your room — that's a Custom View, built exactly the way you need it.

Common Built-in Views

View	Purpose
TextView	Displays text on the screen
EditText	Lets the user type input (like a text box)
Button	A clickable button
ImageView	Displays an image or icon
CheckBox / RadioButton	Lets the user select one or more options
RecyclerView / ListView	Displays a scrollable list of items

Steps to Create a Custom (New) View

1. Create a new Java/Kotlin class that **extends** the `View` class (or an existing view like `TextView`).
2. Override the `onDraw()` method to define exactly what should be drawn on the screen (shapes, colors, custom design).
3. Override `onMeasure()` if you need to control the view's size.
4. Use this new custom view in your XML layout file just like any built-in view.

```
public class CircleView extends View {  
  
    public CircleView(Context context) {  
        super(context);  
    }  
  
    @Override  
    protected void onDraw(Canvas canvas) {  
        super.onDraw(canvas);  
        Paint paint = new Paint();  
        paint.setColor(Color.parseColor("#4299E1"));  
        canvas.drawCircle(100, 100, 80, paint); // draws a blue circle  
    }  
}
```

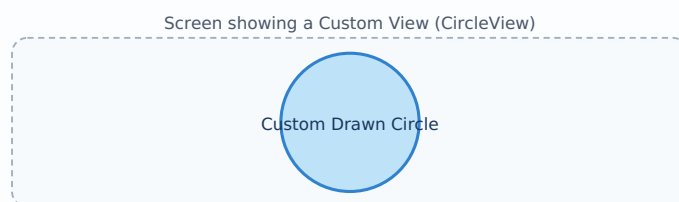


Fig 3.1 — Output of a simple Custom View that draws a circle

DIAGRAM EXPLAINED

- The dashed outer box represents the phone screen (or the area where the view is placed).

- The blue circle inside is exactly what our `onDraw()` code produced — this is the actual visible output the user sees once the Custom View (CircleView) runs.
- Nothing here was drag-and-dropped from the toolbox — every pixel of this circle was manually drawn using code, which is the whole idea of a Custom View.

EXAMPLE

A fitness app showing a **circular progress ring** around your daily step count is usually a **Custom View** — because a plain built-in View cannot draw that special circular design by default.

4. Introduction to Adapters

TECHNICAL TERM

An **Adapter** is a bridge (a middleman) between a data source (like an Array, List, or Database) and a UI component that shows a list of items (like a `ListView`, `RecyclerView`, or `Spinner`). It converts each data item into a View that can be displayed on screen.

SIMPLE ANALOGY

Think of an Adapter like a **waiter in a restaurant**. The kitchen (your data — list of items) prepares the food, but the waiter (Adapter) is the one who actually brings each dish (View) to the customer's table (the screen) one by one.



Fig 4.1 — How an Adapter connects raw data to a visible list on screen

DIAGRAM EXPLAINED

- **Step 1 (green box)** — Your raw data sits somewhere, like an Array, a List, or a Database table.
- **Step 2 (yellow box)** — The Adapter reads this data, one item at a time, and converts each item into a small View (a row layout).
- **Step 3 (blue box)** — The finished rows are handed over to the ListView/RecyclerView, which displays them on the screen as a scrollable list.
- Without the middle "Adapter" step, the data and the screen would have no way to talk to each other.

Common Types of Adapters

Adapter Type	Used For
ArrayAdapter	Simple lists of text data (like an array of strings)
BaseAdapter	Custom, fully controllable list items (used when you need special designs)
CursorAdapter	Displaying data that comes from a database (SQLite) query result
RecyclerView.Adapter	Used with RecyclerView — modern, efficient way to show large scrollable lists

Example: Simple ArrayAdapter

```
String[] fruits = {"Apple", "Banana", "Mango", "Orange"};

ArrayAdapter<String> adapter = new ArrayAdapter<>(
    this,
    android.R.layout.simple_list_item_1,
    fruits
);

ListView listView = findViewById(R.id.listView);
listView.setAdapter(adapter);
```

EXAMPLE

In a Contacts app, you have hundreds of contacts saved (data). The **Adapter** takes each contact's name and photo and turns it into one row you see on the Contacts list screen — repeating this for every single contact.

EXAM TIP

Remember this simple line: **"No Adapter = No List on Screen."** Even if you have data, without an Adapter, a ListView/RecyclerView will show nothing.

5. Intents and Broadcast Receivers — Introduction to Intents

TECHNICAL TERM

An **Intent** is a messaging object used to request an action from another app component. Intents are used to start Activities, start Services, and deliver broadcasts.

SIMPLE ANALOGY

An Intent is like a **request slip** you hand over. If you tell a peon "Please call the Manager," the peon (Android system) understands your request and brings the right result. You don't need to know exactly where the Manager's cabin is — you just describe your intention.

Types of Intents

Type	Meaning	Example
Explicit Intent	You clearly specify which exact Activity/component to open	Opening "SettingsActivity" from "MainActivity" in your own app
Implicit Intent	You describe the action you want; Android decides which app can handle it	Asking to "open a webpage" — Android opens whichever browser app is available

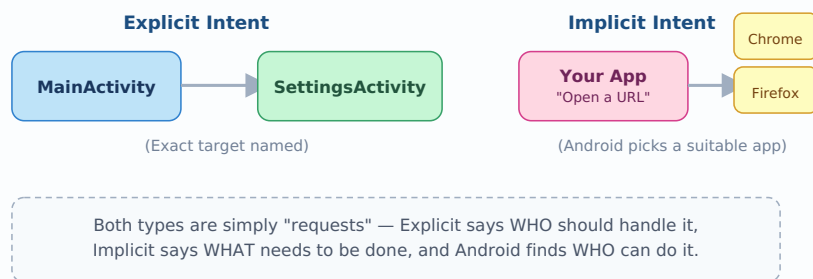


DIAGRAM EXPLAINED

- **Left side (Explicit Intent)** — MainActivity points with a single, direct arrow straight to SettingsActivity. There is no confusion — the exact destination is named in the code.
- **Right side (Implicit Intent)** — "Your App" sends out a request describing only the action ("Open a URL"). Android then looks at all the apps that can perform this action (Chrome, Firefox) and lets the user pick, or opens the default one.
- The bottom dashed box summarises the key difference: Explicit = "I know exactly who should do this," Implicit = "Someone out there can do this, let Android find them."

What Can Intents Do?

- **Start an Activity** – Move from one screen to another.
- **Start a Service** – Begin a background task.
- **Deliver a Broadcast** – Send a system-wide message to all interested apps/components.
- **Pass Data** – Carry small pieces of information (called "Extras") from one screen/app to another.

EXAMPLE

When you tap "Share" on a photo and choose WhatsApp from the list — your app sent an **Implicit Intent** saying "I want to share an image," and Android showed all apps (WhatsApp, Gmail, Instagram) that can handle image sharing.

6. Creating Intents and Broadcast Receivers

A) Creating an Explicit Intent (Moving between screens)

```
// Inside MainActivity.java
Intent intent = new Intent(MainActivity.this, SettingsActivity.class);
intent.putExtra("username", "Rahul"); // passing data
startActivity(intent);
```

Here, `putExtra()` is used to send small pieces of data (like a name) along with the Intent, which the next screen can read.

B) Creating an Implicit Intent (Asking Android to find an app)

```
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("https://www.google.com"));
startActivity(intent); // Opens any available browser app
```

Broadcast Receivers

TECHNICAL TERM

A **Broadcast Receiver** is an Android component that listens for and responds to broadcast messages (Intents) sent by the system or by other apps — even when your app is not currently open.

SIMPLE ANALOGY

A Broadcast Receiver is like a person always listening to a **loudspeaker announcement** in a school. Whenever an announcement matching their interest is made (like "Battery Low" or "SMS Received"), they immediately react to it.

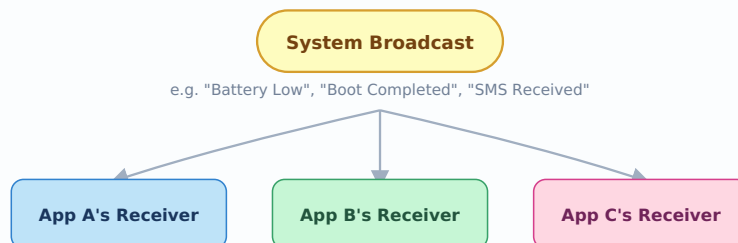


Fig 6.1 — One system broadcast can be received by many apps' Broadcast Receivers at once

DIAGRAM EXPLAINED

- The top yellow bubble represents **one single broadcast announcement** made by the Android system (for example "Battery Low").
- The three arrows show that this **same** announcement travels out to every app that has registered a matching Broadcast Receiver — App A, App B, and App C all hear it at the same time.
- Each app's Broadcast Receiver (the boxes at the bottom) then independently decides what to do with that information — one app might show a warning, another might save data automatically.
- This is different from an Intent that opens one specific Activity — a broadcast can reach many listeners at once, not just one.

Steps to Create a Broadcast Receiver

1. Create a new class that extends `BroadcastReceiver`.
2. Override its `onReceive()` method — this runs when the broadcast is received.
3. Register the receiver — either in the **Manifest file** (static) or in code using `registerReceiver()` (dynamic).

```
public class BatteryReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Battery status changed!", Toast.LENGTH_SHORT).show();
    }
}
```

Registering it in the **AndroidManifest.xml**:

```
<receiver android:name=".BatteryReceiver">
  <intent-filter>
    <action android:name="android.intent.action.BATTERY_LOW" />
  </intent-filter>
</receiver>
```

EXAMPLE

A music app pauses automatically when a phone call comes in. This works because the app has a **Broadcast Receiver** listening for the system's "Incoming Call" broadcast.

EXAM TIP

Remember: **Intent** = the message/request itself. **Broadcast Receiver** = the listener that reacts when that message arrives. They always work as a pair.

7. Using Internet Resources — Downloading and Parsing

Most modern apps need data from the internet — like weather updates, news, or images. This means the app has to **download** data from a server and then **parse** (understand/read) it.

TECHNICAL TERMS

Downloading = Getting raw data (text, images, files) from the internet onto the device.

Parsing = Reading and converting that raw downloaded data (usually in JSON or XML format) into a form the app's code can actually use.

SIMPLE ANALOGY

Downloading is like **receiving a parcel** from the courier. Parsing is like **opening the parcel and reading the instructions inside** to understand what's in it and how to use it.

How Data Flows from the Internet into Your App



Fig 7.1 — Journey of data from the Internet to your App's screen

DIAGRAM EXPLAINED

- **Step 1 — Download:** The app connects to the Web Server and downloads the Raw Data (usually in JSON or XML format, shown as plain text at this stage).
- **Step 2 — Parse:** That raw text is read and converted into proper Java/Kotlin Objects that the app's code can actually understand and work with (like a "User" object with a name and age).
- **Step 3 — Display:** Finally, those objects are used to fill in the app's screen — for example, showing the user's name in a TextView.
- Each arrow is a separate stage — skipping the "Parse" step would mean the app has data, but no usable way to read it.

Common Data Formats

Format	Meaning
JSON (JavaScript Object Notation)	Lightweight, easy-to-read text format using key-value pairs — most commonly used today
XML (Extensible Markup Language)	Older tag-based format, still used by some services

Example: A Simple JSON Response

```
{
  "name": "Rahul",
  "age": 21,
  "city": "Ahmedabad"
}
```

Example: Parsing JSON in Android (using org.json)

```
String jsonString = "{ \"name\": \"Rahul\", \"age\": 21 }";

JSONObject obj = new JSONObject(jsonString);
String name = obj.getString("name"); // Rahul
int age = obj.getInt("age"); // 21
```

EXAMPLE

A Weather app sends a request to a weather server. The server sends back JSON data like temperature and humidity. The app **downloads** this JSON and then **parses** it to show "32°C, Sunny" on the screen in a nice, readable format.

NOTE

Since Android 9, apps cannot directly access plain **HTTP** (unsecured) internet connections by default — **HTTPS** (secure) is required for safety, unless specially allowed in the Manifest.

8. Using the Download Manager

TECHNICAL TERM

DownloadManager is a built-in Android system service that handles long-running HTTP downloads in the background, automatically managing retries, network changes (Wi-Fi/Mobile Data), and showing download progress — without the developer needing to write all that logic manually.

SIMPLE ANALOGY

Instead of you personally standing at the door waiting for a parcel (manually managing the download), you hire a **courier company (DownloadManager)** that handles pickup, delivery, retries if it fails, and even notifies you when it's done.

Why Use DownloadManager Instead of Writing Your Own Code?

- **Handles Big Files** – Good for large files like PDFs, videos, APKs.
- **Runs in Background** – The download continues even if the user leaves the app.
- **Automatic Retry** – If internet disconnects, it resumes automatically when reconnected.
- **Shows Progress** – Displays a notification with download progress automatically.
- **Saves Developer Time** – No need to manually manage threads, notifications, or retries.

Steps to Use DownloadManager



Fig 8.1 — Steps to download a file using Android's DownloadManager

DIAGRAM EXPLAINED

- **Step 1 — Create a Request object:** You create a `DownloadManager.Request` and tell it which file's URL you want to download.
- **Step 2 — Set URL & save location:** You specify exactly where the internet file is, and where on the phone it should be saved once downloaded.
- **Step 3 — Enqueue the request:** You hand this request over to the system using `enqueue()` — this is like placing your order and stepping back.
- **Step 4 — System handles it:** From here on, Android itself manages the download in the background, retries on failure, and shows a notification when it's finished — the app doesn't need to keep checking on it.

Example Code: Downloading a File

```

DownloadManager.Request request =
    new DownloadManager.Request(Uri.parse("https://example.com/file.pdf"));

request.setTitle("Downloading File");
request.setDescription("Please wait...");
request.setNotificationVisibility(
    DownloadManager.Request.VISIBILITY_VISIBLE_NOTIFY_COMPLETED);
request.setDestinationInExternalPublicDir(
    Environment.DIRECTORY_DOWNLOADS, "myfile.pdf");

DownloadManager manager =
    (DownloadManager) getSystemService(Context.DOWNLOAD_SERVICE);
manager.enqueue(request);
  
```

EXAMPLE

When you tap "Download" on a PDF notes file from a college app, the phone shows a progress notification and the file lands in your **Downloads** folder — this is exactly the **DownloadManager** in action.

9. Using Internet Services

TECHNICAL TERM

An **Internet Service** (Web Service / API) is a program running on a remote server that an app can talk to over the internet to get or send data, using standard rules like **REST** or **HTTP** requests.

SIMPLE ANALOGY

A web service is like a **waiter taking your order to the kitchen** in a restaurant and bringing back the food. Your app is the customer, the internet service is the kitchen, and the request/response is the order and the dish.

Common Ways to Use Internet Services in Android

Method / Library	Simple Explanation
HttpURLConnection	The basic, built-in Java way to send/receive data over the internet (a bit manual)
Volley	A Google library that makes network requests and caching simpler and faster
Retrofit	A very popular modern library that makes calling web APIs simple and clean
OkHttp	A powerful and efficient HTTP client, often used along with Retrofit

HTTP Request Methods (Basic Vocabulary)

Method	Purpose
GET	Ask the server to send/return data (e.g., fetch a list of products)
POST	Send new data to the server (e.g., submit a signup form)
PUT	Update existing data on the server
DELETE	Remove data from the server



Fig 9.1 — App communicating with a Web (Internet) Service

DIAGRAM EXPLAINED

- **Top arrow (Request):** The Android App (Client) sends a request to the Web Service — this request says what is needed, using a method like GET or POST.
- **Bottom arrow (Response):** The Web Service processes that request and sends back a Response, usually as JSON data, which the app then reads and displays.
- This "ask and answer" pattern repeats every time the app needs fresh data — each request gets exactly one response back.

Example: Simple GET Request using Retrofit

```

public interface ApiService {
    @GET("users/{id}")
    Call<User> getUser(@Path("id") int userId);
}

Retrofit retrofit = new Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build();
  
```



```
ApiService api = retrofit.create(ApiService.class);
Call<User> call = api.getUser(1);

call.enqueue(new Callback<User>() {
    @Override
    public void onResponse(Call<User> call, Response<User> response) {
        User user = response.body(); // Data received!
    }
    @Override
    public void onFailure(Call<User> call, Throwable t) {
        // Handle error
    }
});
```

EXAMPLE

A shopping app calls an internet service to **GET** the list of products, and calls another internet service to **POST** your order details when you click "Buy Now".

IMPORTANT NOTE

Just like the internet itself, network operations must never run on the **Main/UI Thread** in Android (it will freeze the app). They must run on a background thread — most modern libraries like Retrofit handle this automatically.

✓ Quick Summary (Revision Page)

- **Layout** = Invisible container that arranges Views on screen (LinearLayout, RelativeLayout, ConstraintLayout, etc.)
- **Fragment** = A reusable mini-screen that lives inside an Activity; has its own lifecycle.
- **Custom View** = A new View created by extending the View class, used when built-in views aren't enough.
- **Adapter** = A bridge that converts data into Views for lists (ArrayAdapter, RecyclerView.Adapter, etc.)
- **Intent** = A message/request to start an Activity, Service, or send a Broadcast (Explicit vs Implicit).
- **Broadcast Receiver** = A listener that reacts to system-wide or app broadcasts (like Battery Low, SMS Received).
- **Downloading & Parsing** = Getting raw internet data (JSON/XML) and converting it into usable app data.
- **DownloadManager** = Built-in Android system service for managing background file downloads automatically.
- **Internet Services** = Talking to remote web servers/APIs using GET, POST, PUT, DELETE via tools like Retrofit, Volley, OkHttp.

Prepared as an easy study guide for MCA Semester 3 — Unit 2: User Interface in Android • For classroom and exam revision use